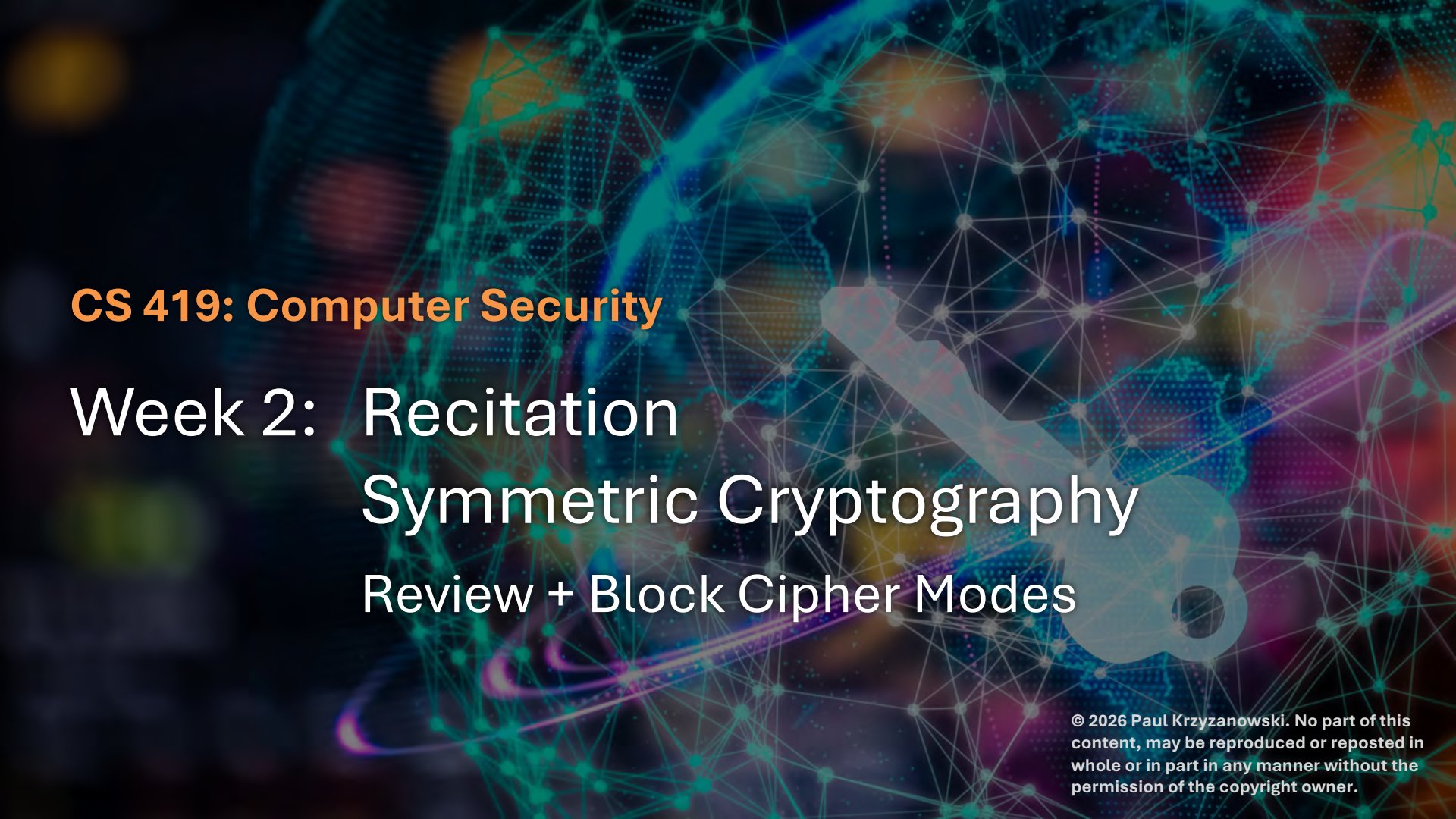


CS 419: Computer Security

Week 2: Recitation
Symmetric Cryptography
Review + Block Cipher Modes

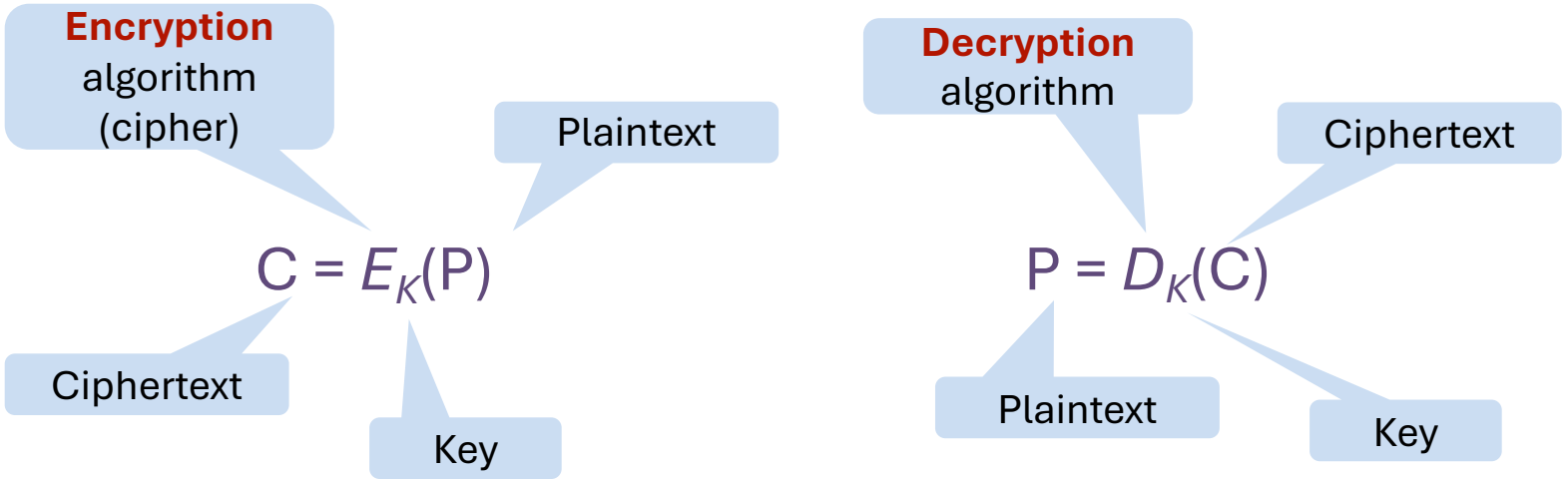
© 2026 Paul Krzyzanowski. No part of this content, may be reproduced or reposted in whole or in part in any manner without the permission of the copyright owner.

Review

Core Concepts

Symmetric Cryptography

Symmetric cryptography: one shared secret key, K, for encryption & decryption



Keyspace = set of all keys

Kerckhoffs's principle: the system must remain secure even if the attacker knows everything about it except the key

Schneier's law: anyone can design a cipher they themselves cannot break – it needs years of failed attacks by other people

Classical Ciphers

- **Monoalphabetic substitution ciphers**
  - One character in plaintext always produces the same ciphertext character

    `ABCABC → QMVQMV`

- **Polyalphabetic substitution ciphers**
  - The same character in plaintext may map to different characters in ciphertext

    `ABCABC → QMVGRS`

- **Transposition ciphers**
  - The characters are shuffled around but not substituted

    `ABCABC → BACCBA`

How the Classical Ciphers Were Broken

- Every classical cipher was broken the same way:
  a pattern in the plaintext survived encryption
- Monoalphabetic substitution: letter frequencies survive.
  - Frequency analysis enables guessing the key from the ciphertext
- Vigenère: the keyword repeats.
  - Repeated plaintext under the same part of the keystream gives repeated ciphertext
- Enigma:
  - Predictable messages and a machine that never mapped a letter to itself

Any pattern that survives encryption is the start of an attack

Shannon Entropy in Cryptography

Definition

- A measure of randomness in a dataset
- High entropy in ciphertext and keys ensures unpredictability, making attacks harder

Formula: $H(X) = -\sum P(x) \log_2 P(x)$

- H(x) = Entropy measured in bits
- P(x) = Probability of symbol x occurring in the message

Note: you don't need to know the formula – just know what entropy is

Results

- Low entropy → Predictable data (easier to attack)
- High entropy → Random, unpredictable data (what we want for cryptography)
  - High entropy in ciphertext makes it statistically indistinguishable from random noise
- Polyalphabetic ciphers increase entropy

Perfect Secrecy and the One-Time Pad

Perfect secrecy: the ciphertext reveals nothing about the plaintext. The one-time pad achieves it.

```
Plaintext    1 0 1 1 0 0 1 0
Random key   0 1 1 0 1 1 0 0
Ciphertext   1 1 0 1 1 1 1 0
```

Three conditions must hold

1. The key is truly random.
2. The key is as long as the message.
3. The key is never used again.

`Use the same key twice and it cancels:  C1 XOR C2 = P1 XOR P2`

The XOR of two English messages is enough to recover both. Key reuse breaks the pad, and it will come back today.

Perfect but impractical: every byte of traffic consumes a byte of key

Confusion & Diffusion

Two fundamental principles for designing secure ciphers

- They help hide the relationship between plaintext, ciphertext, and the encryption key
- Make attacks more difficult

Confusion – *hides the effect of the key on ciphertext*

- **Makes the relationship between the encryption key and ciphertext as complex as possible**
- Prevents attackers from determining the key even if they analyze multiple ciphertexts
- Effect: small changes in the key cause unpredictable changes in the ciphertext

Diffusion – *small changes in plaintext are spread throughout the ciphertext*

- Spreads out plaintext influence over the entire ciphertext
- Ensures that changing a single plaintext bit affects many bits of ciphertext
- Effect: Prevents frequency analysis and patterns appearing in ciphertext

Block Ciphers and Stream Ciphers

Block cipher

Encrypts one fixed-size block, 128 bits for AES, into a block of the same size.

Ciphertext is built from rounds of substitution and mixing: confusion and diffusion.

Stream cipher

Generates a keystream from a key and a nonce, a number used once, and XORs it with the plaintext: a one-time pad with a computed key.
ChaCha20 is the one in use.
The nonce must never repeat under a key.

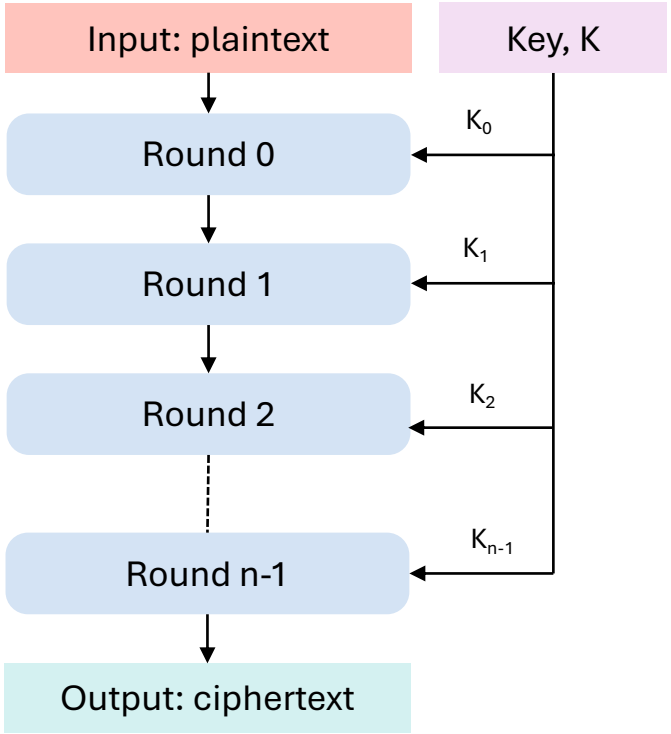
AES: 128-bit block, 128- to 256-bit key, 10 to 14 rounds. A quarter century of analysis and no practical attack.

Keys and nonces come from the operating system's random number generator. Predictable random values break everything above.

A block cipher encrypts 16 bytes. Almost nothing you want to encrypt is 16 bytes.

Block ciphers

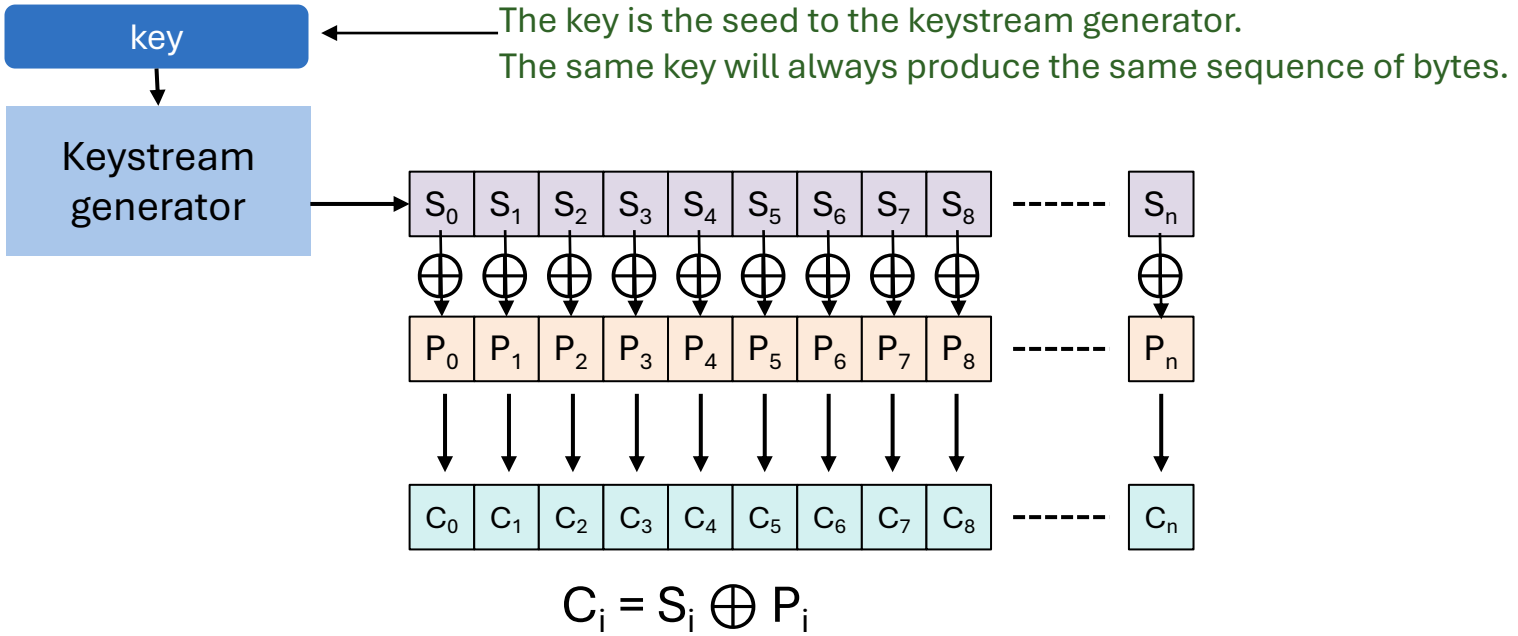
- Block ciphers encrypt a *block* of plaintext at a time
- DES & AES are two popular block ciphers
  - DES: uses 64-bit blocks
  - AES: uses 128-bit blocks
- Block ciphers are **iterative ciphers**
  - The encryption process is an iteration through several *round* operations
  - A single round does not provide perfect confusion or diffusion
  - Each round uses a subkey derived from the key
    - Determines what substitutions & transpositions take place



Stream ciphers (e.g., ChaCha20)

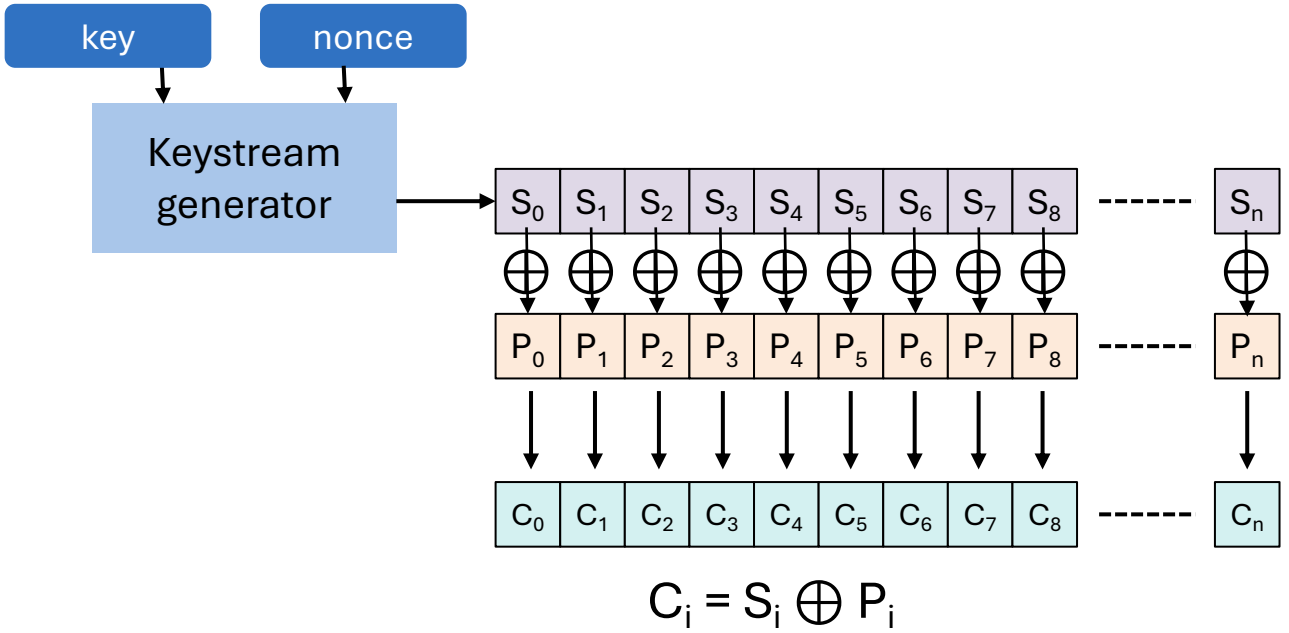
Key stream generator produces a sequence of pseudo-random bytes

Simulates a one-time pad



Stream ciphers (e.g., ChaCha20)

Reusing the same keystream breaks security, so we initialize the keystream generator with the user's key AND a random value (called a *nonce*)



Encrypting more than one block

Cipher modes define how block ciphers (e.g., AES) encrypt data larger than a single block (typically 128 bits)

153 Million Passwords Stolen

October 2013: Attackers copy a database of Adobe user records

- The passwords are encrypted with Triple DES, a cipher that nobody has broken
- Within days, the most common password is known: 123456
  - Then the second most common, the third, and every account that shares them
- Nobody broke the cipher. Nobody had to.

Encrypting Block by Block

The obvious way to encrypt a list of passwords: cut it into blocks and encrypt each block with the key.



Identical plaintext blocks produce identical ciphertext blocks.

The contents of each block are hidden. The pattern of blocks is not.

At Adobe, one encrypted value appeared 1.9 million times.
It did not take a cryptanalyst to guess what it was.

Two equal inputs give two equal outputs: a regularity, and regularities are where attacks start

What Block-by-Block Encryption Leaks

Patterns

Equal blocks show as equal blocks.
An encrypted image keeps its outlines.
A database column reveals which rows match.

Structure

Blocks can be deleted, duplicated, or reordered, and the receiver cannot tell.
Swap two encrypted salary records and both decrypt fine.

Neither problem is in the cipher.
Block ciphers were designed to encrypt one block at a time.

What we need from a way of applying a block cipher to a whole message:

1. The same message encrypted twice with the same key should give different ciphertext.
2. Equal blocks inside a message should not show as equal.
3. Ideally, tampering with the ciphertext should be detectable.

Modes of Operation

A **mode of operation** is the procedure for applying a block cipher to a message longer than one block

- The cipher and the mode are separate choices.
  - AES can be used in any mode
- The mode determines whether:
  - Patterns show among blocks
  - Blocks can be rearranged by an attacker
  - Tampering is detected
- Most modes take a public per-message value that changes every time.
  - A **nonce** must never repeat with the same key
  - An **initialization vector** (**IV**) starts the first block

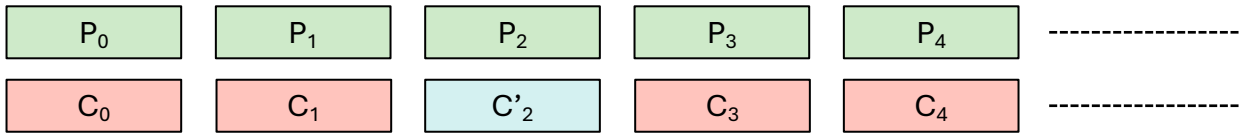
It is not a good idea to use block ciphers directly

With block ciphers, a stream of data is broken into *k*-byte blocks

- Each block is encrypted separately
- This is called **Electronic Codebook** (**ECB**)

Problems

1. The same plaintext in the data results in identical encrypted blocks
   - Attackers can build a table (codebook) of plaintext/ciphertext matches
   - Even without decrypting, attackers can see patterns of identical blocks
2. Attackers can add/delete/replace blocks from other messages encrypted with the same key and you may not know
3. A direct relationship between plaintext & ciphertext helps cryptanalysis
   - The same content in multiple messages encrypted with the same key will have the same ciphertext



Goal of block cipher modes

Hide any patterns in encrypted data when encrypting multiple blocks

We do this by:

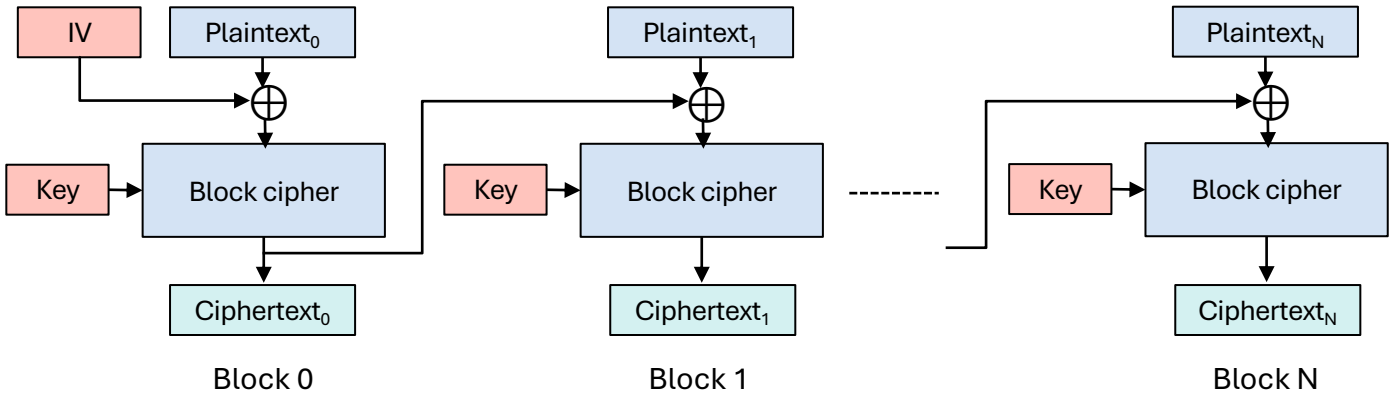
1. Transforming each block so that its ciphertext is dependent on its position in the input stream
2. Adding random initialization data for each encryption session

We will only look at three modes as examples – there are more

Cipher Block Chaining (CBC) mode

- Random initialization vector (IV) = bunch of *k* random bits
  - Non-secret: both parties need to know this – usually added to the start of the message
- Exclusive-or the IV with first plaintext block – then encrypt the block
- For every other block, XOR the ciphertext of the previous block with the plaintext

$$c_i = E_K(m_i) \oplus c_{i-1}$$



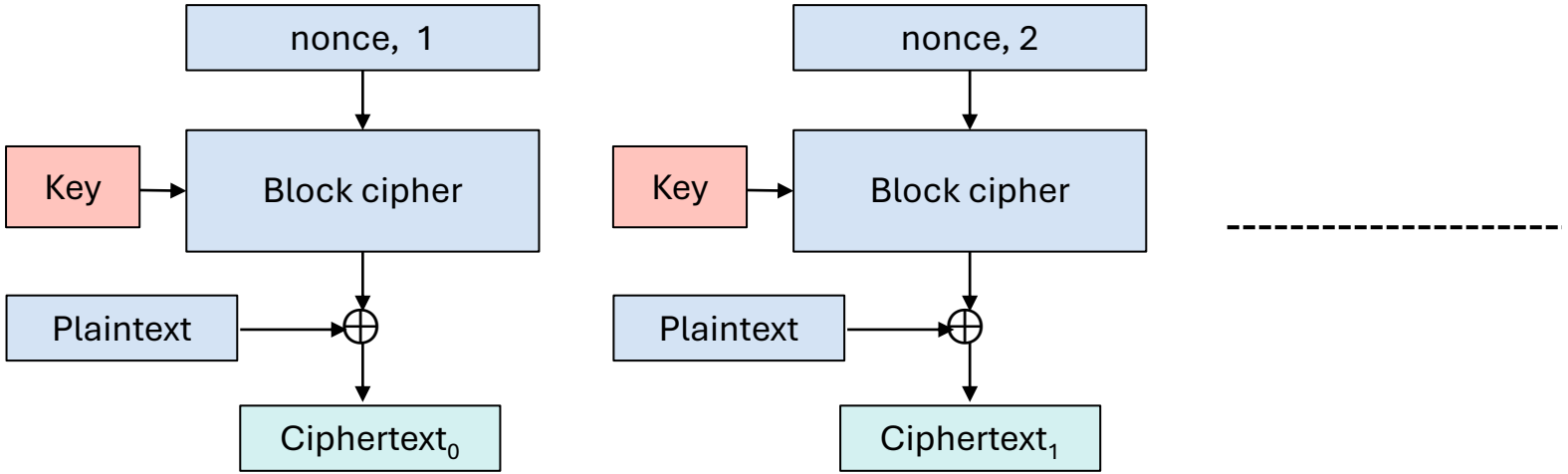
CBC Drawbacks

- The good
  - Identical plaintext does not produce the same ciphertext
    - Unless all previous blocks and the initialization vector are identical
  - Each block is a function of all previous blocks
    - Great for diffusion: changes in one block cause all future blocks to change
- The problems
  1. Encryption cannot run in parallel. Each block needs the previous ciphertext block first
  2. The last block must be filled out with padding and removed with decryption
  3. No tamper detection.
     - Flip one bit of C_1: P_1 decrypts to garbage
     - But $P_2 = D(C_2)$ XOR C_1, so the same bit flips in P_2

Block encryption: Counter (CTR) mode

- Random starting counter or a nonce with a count appended to it
  - Any function producing a non-repeating sequence (an incrementing number is a common function)
- **Encrypt the counter with the key**
- Exclusive-or result with plaintext block

CTR makes a block cipher behave like a stream cipher

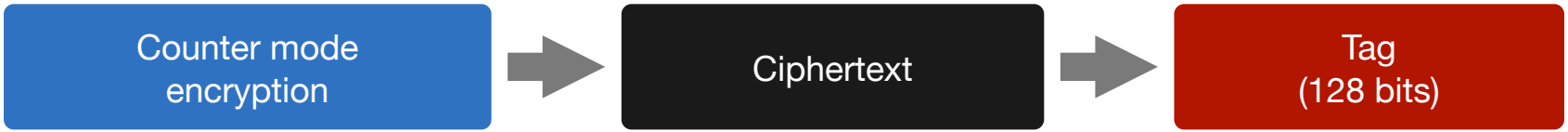


CTR Mode Observations

- Supports parallel processing
  - One block does not depend on the results of the previous blocks
- Nonce reuse is a problem
  - It will allow attackers to identify matching blocks from other ciphertext encrypted with the same key – same weakness as reusing keys in a one-time pad
  - 2017, KRACK: the attack on WPA2 Wi-Fi tricked clients into reinstalling a key, which reset the nonce counter
- Note: ECB, CBC, nor CTR mode do not provide integrity or authentication
  - A receiver can't tell if any ciphertext has been modified
  - We'll look at integrity later
- **Malleability**: ciphertext is *plaintext* XOR *keystream*.
  - Flip one ciphertext bit, and that same plaintext bit flips. No garbage, nothing to notice

Galois/Counter Mode

GCM: counter mode, plus a short check value computed over the ciphertext and any headers.



The receiver recomputes the tag from the ciphertext it received.
If it does not match, the message is rejected unread.

Any change to the ciphertext, the nonce, or the headers changes the tag.
Bit flipping no longer works.

The tag is computed with fast polynomial arithmetic.

Requirements: a 96-bit nonce that never repeats with the same key.
Nonce reuse leaks plaintext and lets an attacker forge tags.

The default mode in TLS 1.3, the protocol behind nearly every https connection

Authenticated Encryption

Authenticated encryption with associated data (AEAD):

- Encryption that also produces a tag, and decryption that refuses anything whose tag fails

- **Inputs**: key, nonce, plaintext, and **associated data**, headers that stay readable but are covered by the tag

- **Output**: ciphertext plus tag.
  - Any change to any input makes verification fail

- The receiver never sees plaintext from a modified message
  - AES-GCM and ChaCha20-Poly1305 are the two in use
  - TLS 1.3 requires encryption with AEADs

Choosing a Mode

The four modes, what each needs per message, and what each detects.

Mode	Idea	Per-message input	Tamper	Use
ECB	Each block alone	None	No	Single blocks only
CBC	Chain to previous block	Unpredictable IV	No	Legacy, with a separate check
CTR	Encrypt counters, XOR	Nonce, never repeated	No	Building block
GCM	CTR plus a tag	Nonce, never repeated	**Yes**	New systems

New system designs: use an AEAD
AES-GCM on systems with AES hardware, ChaCha20-Poly1305 without.
Let the library generate the nonces.

Where Modern Encryption Fails

- ECB where a real mode was needed: Adobe, 2013
- A nonce that repeated:
  - KRACK, 2017, and every system restored from the same virtual machine snapshot
- Ciphertext that was never authenticated:
  - Bit flipping on CTR, padding attacks on CBC
- Not one of these is a weakness in the cipher
- Use a well-reviewed library's AEAD interface, let it generate the nonces, and spend the effort on managing the keys

The End